

Rajalakshmi Engineering College

Name: Harsan J
Email: 240701179@rajalakshmi.edu.in
Roll no: 240701179
Phone: 8925466195
Branch: REC
Department: CSE - Section 10
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Raj is solving a physics problem involving projectile motion, where he needs to calculate the time a ball hits the ground using a quadratic equation of the form $ax^2 + bx + c = 0$. Depending on the coefficients, the ball may hit the ground once, twice, or not at all in real time.

Help Raj find all real roots of the equation, if any.

Note: discriminant = $b^2 - 4ac$

Input Format

The input consists of three space-separated doubles a, b, and c, representing the coefficients of the quadratic equation.

Output Format

If there are two real roots, print:

- "Two real solutions:"
- "Root1 = <value>"
- "Root2 = <value>"

If there is one real root, print:

- "One real solution:"
- "Root = <value>"

If there are no real roots, print:

- "There are no real solutions."

Note: values are rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 6 9

Output: One real solution:

Root = -3.00

Answer

```
// You are using Java
import java.util.Scanner;
class QuadraticEquation
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        double a = sc.nextDouble();
        double b = sc.nextDouble();
        double c = sc.nextDouble();
        double discriminant = b * b - 4 * a * c;
        if (discriminant > 0)
        {
            double root1 = (-b + Math.sqrt(discriminant)) / (2 * a);
```

```

double root2 = (-b - Math.sqrt(discriminant)) / (2 * a);
System.out.println("Two real solutions:");
System.out.printf("Root1 = %.2f%n", root1);
System.out.printf("Root2 = %.2f%n", root2);
} else if (discriminant == 0)
{
    double root = -b / (2 * a);
    System.out.println("One real solution:");
    System.out.printf("Root = %.2f%n", root);
} else
{
    System.out.println("There are no real solutions.");
}
sc.close();
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ram wants to evaluate the time required to break even on an investment based on initial costs, monthly profits, and monthly expenses. Write a program to calculate the break-even point in months and categorize the return on investment.

Compute the break-even point by using the formula: $\text{initial cost} / (\text{monthly profit} - \text{monthly expenses})$. Based on the break-even point, classify the return on investment into one of the following categories: Quick Return: If the break-even point is 3 months or fewer. Average Return: If the break-even point is between 4 and 12 months, inclusive. Long-term Return: If the break-even point exceeds 12 months.

Ram is new to programming, so he seeks your assistance in creating the program.

Note: monthly profit is always greater than monthly expenses.

Input Format

The first line of input consists of a double value representing the initial cost.

The second line consists of a double value representing the monthly profit.

The third line consists of a double value representing the monthly expenses.

Output Format

The first line prints "Break-even Point:", followed by the break-even point as a decimal number (of double datatype), formatted to two decimal places.

The second line prints "Category: ", followed by the investment return as a String, which can be one of:

- "Quick Return" if break-even point ≤ 3
- "Average Return" if break-even point ≤ 12
- "Long-term Return" if break-even point > 12

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10000.50

5000.75

1000.10

Output: Break-even Point: 2.50

Category: Quick Return

Answer

```
// You are using Java
import java.util.Scanner;
class Main
{
    public static void main(String[] args)
    {
        Scanner scan=new Scanner(System.in);
        double ini=scan.nextDouble();
        double pro=scan.nextDouble();
        double exp=scan.nextDouble();
        double brek=ini/(pro-exp);
        System.out.printf("Break-even Point: %.2f\nCategory: ",brek);
        if(brek<=3)
```

```
{
    System.out.print("Quick Return");
}
else if(brek<=12 && brek>=4)

{
    System.out.print("Average Return");
}
else
{
    System.out.print("Long-term Return");
}
}
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Ted, the computer science enthusiast, has accepted the challenge of writing a program that checks if the number of digits in an integer matches the sum of its digits.

Guide Ted in designing and writing the code to solve this problem using a 'do-while' loop.

Input Format

The input consists of an integer N, representing the number to be checked.

Output Format

If the sum is equal to the number of digits, print "The number of digits in N matches the sum of its digits."

Else, print "The number of digits in N does not match the sum of its digits."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 20

Output: The number of digits in 20 matches the sum of its digits.

Answer

```
// You are using Java
import java.util.Scanner;
class Main
{
    public static void main(String[] args)
    {
        Scanner scan=new Scanner(System.in);
        String a=scan.nextLine();
        int len=a.length();
        int count=0;
        for(int i=0;i<a.length();i++)
        {
            char ch=a.charAt(i);
            int num=ch-'0';
            count+=num;
        }
        if(count==len)
        {
            System.out.printf("The number of digits in %s matches the sum of its
digits.",a);
        }
        else
        {
            System.out.printf("The number of digits in %s does not match the sum of
its digits.",a);
        }
    }
}
```

}
Status : Correct

Marks : 10/10

4. Problem Statement

Noah is analyzing numbers within a given range $[A, B]$ and wants to calculate a special sum. For each number in the range, he calculates the product of its odd digits (ignoring even digits). If the number contains no odd digits, it is skipped. The sum of these products for all numbers in the range is the result.

Write a program to compute this sum.

Example

Input:

10 12

Output:

3

Explanation:

For 10, odd digits = 1, product = 1.

For 11, odd digits = 1, 1, product = $1 * 1 = 1$.

For 12, odd digits = 1, product = 1.

Total sum = $1 + 1 + 1 = 3$

Input Format

The input consists of two space-separated integers A and B, representing the inclusive range boundaries.

Output Format

The output prints a single integer representing the sum of the products of odd digits for all numbers in the range.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10 12

Output: 3

Answer

```
// You are using Java
import java.util.Scanner;
class OddDigitProductSum
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int A = sc.nextInt();
        int B = sc.nextInt();
        int totalSum = 0;
        for (int num = A; num <= B; num++)
        {
            int temp = num;
            int product = 1;
            boolean hasOdd = false;
            while (temp > 0)
            {
                int digit = temp % 10;
                if (digit % 2 != 0)
                {
                    product *= digit;
                    hasOdd = true;
                }
                temp /= 10;
            }
            if (hasOdd)
            {
                totalSum += product;
            }
        }
    }
}
```



```
        System.out.println(totalSum);  
        sc.close();  
    }  
}
```

Status : Correct

Marks : 10/10